DevOps Fundamentals Assignment 1

Digital Banking Transformation

A DevOps transformation roadmap for a traditional bank

Field	Details
Student name	Mohit
Institution	National Institute of Information Technology
Course / Subject	Java Full stack
Instructor	Mr. Pravin
Submission date	October, 3, 2026

Table of Contents

Table of Contents 2
1. Executive Summary 3
2. Business Scenario and Objectives 3
2.1 Scenario 3
2.2 Objectives 3
3. Stakeholders 3
4. Current State, Challenges and Risks 4
4.1 Current State Analysis 4
4.2 Challenges 4
4.3 Risks of the Transformation Itself 5
5. Assumptions and Constraints 5
5.1 Given Requirements (from the assignment) 5
5.2 Assumptions (made by the author) 6
5.3 Constraints 6
6. Proposed Solution: DevOps Transformation Roadmap 6
6.1 Vision and Guiding Principles 6
6.2 Collaboration Practices 7
6.3 Automation Opportunities 7
6.4 CI/CD Concepts and Pipeline 8
6.5 Quality Gates 10
6.6 Monitoring Strategy 10
6.7 Feedback Loops 11
6.8 Cultural Changes 11
7. Implementation Roadmap 12
7.1 Release Cadence Milestones (illustrative) 13
8. Measurable Success Criteria 13
9. Recommendations and Lessons Learned 13
9.1 Recommendations 14
9.2 Lessons Learned 14
9.3 Limitations of This Plan 14

1. Executive Summary

Problem. A traditional bank releases its applications only every six months. Releases rely on manual testing and manual deployment, teams work separately, and many releases fail. Customers wait a long time for improvements and each release is a high-stress event.

Solution. An 18-month DevOps transformation roadmap built on six ideas: shared team ownership, automation of testing and deployment, a CI/CD pipeline, automated quality gates, monitoring, and fast feedback loops. It starts with a pilot application and a culture change, not with tools.

Expected result. Smaller, safer and more frequent releases (from every six months to monthly, then every two weeks for digital channels), fewer failures, faster recovery, and a complete audit trail that satisfies banking regulation. Section 8 defines how this will be measured.

2. Business Scenario and Objectives

2.1 Scenario

The bank deploys applications every six months. Each release bundles months of changes, is tested by hand, and is deployed by hand. Development, testing, operations and security teams are disconnected, so work is passed between them like a relay race. Failure rates are high.

Simple explanation of DevOps: DevOps is a way of working in which the people who build software and the people who run it work as one team, using automation to deliver small changes quickly and safely.

2.2 Objectives

#	Objective	Why it matters
O1	Release more often, in small changes	Customers get improvements sooner; each change is less risky
O2	Reduce release failures	Failures cost money, trust and weekends
O3	Automate testing and deployment	Manual work is slow and error-prone
O4	Break down silos between teams	Shared ownership removes hand-off delays and blame
O5	Keep security and compliance strong	A bank must be safe and auditable at higher speed
O6	See problems quickly and recover fast	Monitoring and rollback limit customer impact
O7	Build a culture of learning	Tools fail without the right habits and attitude

3. Stakeholders

A stakeholder is anyone affected by, or responsible for, the transformation.

Stakeholder	Role	What they need
Executive leadership (CIO, CTO)	Sponsor and fund the change	Business value, controlled risk, visible progress
Business and product owners	Decide features and priorities	Faster delivery of customer features
Development teams	Build the applications	Fast feedback, less rework, modern tools
QA / Testers	Check quality	Automation for repetitive tests; time for exploratory testing
Operations / Infrastructure	Run production systems	Stable, repeatable deployments; fewer midnight emergencies

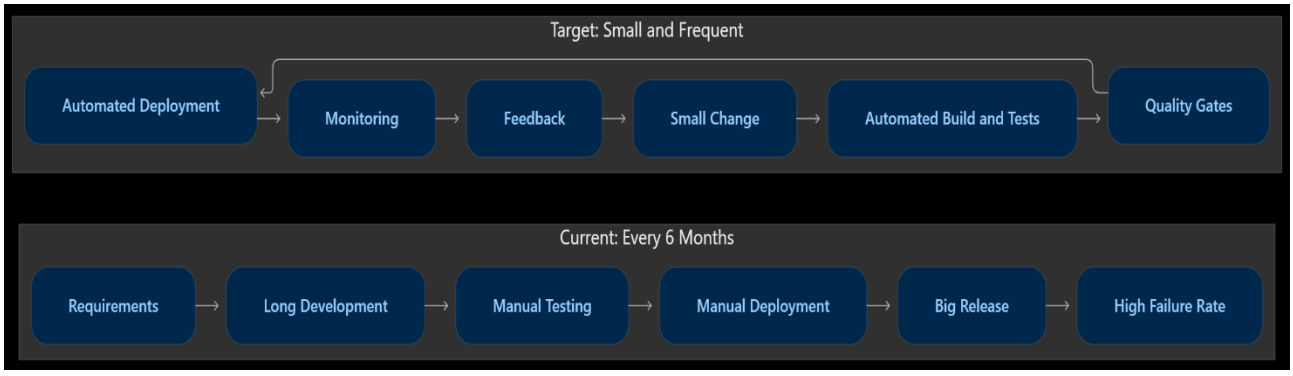
Stakeholder	Role	What they need
Information Security	Protect data and systems	Security checks built into the pipeline
Risk, Compliance and Audit	Ensure rules are followed	Traceable, evidence-backed changes
Change Advisory Board (CAB)	Approves production changes	Clear risk information; a smoother approval process
Regulators (external)	Supervise the bank	Confidence that controls remain strong
Customers	Use digital banking	Reliable service and steady improvements

4. Current State, Challenges and Risks

4.1 Current State Analysis

Area	Current state (from the assignment)	Problem it causes
Release frequency	Every six months	Huge batches of changes; customers wait; high risk per release
Testing	Manual	Slow, inconsistent, covers less; bugs found late
Deployment	Manual	Human error, long downtime windows, hard to repeat
Team structure	Disconnected teams	Hand-offs, delays, finger-pointing, no shared goals
Outcome	High failure rates	Outages, rollbacks, loss of customer and business trust

Why big releases fail (a vicious cycle). When a release contains hundreds of changes, nobody can tell which one broke something. Because releases are risky, the bank releases less often. That makes the next release even bigger and riskier. DevOps breaks this cycle by making releases small and frequent, so each one is easy to test, understand and undo.



4.2 Challenges

#	Challenge	Impact	Risk
C1	Six-month release cycle	Slow delivery; large, risky releases	High
C2	Manual testing	Late bug discovery; inconsistent quality	High
C3	Manual deployment	Errors, downtime, no repeatability	High
C4	Disconnected teams and hand-offs	Delays and blame; nobody owns the whole result	High
C5	High failure rate	Outages and customer impact	High

#	Challenge	Impact	Risk
C6	Slow recovery (no monitoring or automated rollback)	Long outages; reputation damage	High
C7	Compliance checks done by paperwork at the end	Slow approvals; evidence hard to produce	Medium
C8	Security tested late	Vulnerabilities reach production	High
C9	Knowledge held by a few people	Single points of failure; slow onboarding	Medium

Risk level is a judgement based on the likely effect on a bank (money, customers, regulation). It is not measured data.

4.3 Risks of the Transformation Itself

Changing how a bank works carries its own risks. They are listed here so they can be managed, not ignored.

#	Risk	Level	Mitigation
T1	Staff resist change or fear job loss	High	Leadership sponsorship; training; involve staff early; start with volunteers
T2	Speed weakens compliance or security	High	Build controls and evidence into the pipeline; involve Compliance and Security from day one
T3	Legacy systems (for example core banking) cannot be automated quickly	High	Start with digital channels; treat legacy separately; use APIs and test automation around it
T4	Tool-first approach without culture change	High	Change practices and habits first; tools support them
T5	Too many tools, high cost	Medium	Choose a small standard toolset; review cost after the pilot
T6	Skills gap	Medium	Training, coaching, DevOps champions, hiring where needed
T7	Early incidents damage confidence	Medium	Pilot on a low-risk application; canary releases; fast rollback

5. Assumptions and Constraints

This section separates what the assignment **requires** from what this report **assumes**, so the two are never confused.

5.1 Given Requirements (from the assignment)

#	Requirement
R1	A traditional bank deploys applications every six months
R2	Releases involve manual testing, manual deployments, disconnected teams and high failure rates
R3	Analyse the current state
R4	Design a DevOps transformation roadmap
R5	Include collaboration practices
R6	Include automation opportunities
R7	Include CI/CD concepts
R8	Include quality gates
R9	Include a monitoring strategy

#	Requirement
R10	Include feedback loops
R11	Include cultural changes
R12	Deliver a report, workflow diagrams, recommendations, lessons learned and an implementation plan

5.2 Assumptions (made by the author)

#	Assumption	Effect if wrong
A1	The bank is mid-sized, with a few hundred IT staff and a portfolio of customer-facing and internal applications	Timeline and team structure would change
A2	Digital channels (internet banking, mobile, notifications) can be modernised first; core banking systems are older and slower to change	Roadmap scope and order would change
A3	Senior leadership supports the transformation and funds it	Without sponsorship the plan is unlikely to succeed
A4	Regulators accept frequent releases if controls, approvals and audit trails are maintained	Release cadence targets may need adjusting
A5	The assignment gives only the six-month cycle; all other baseline figures (failure rate, recovery time, test automation) are **assumed** and illustrative	Targets in Section 8 must be re-based on real measurements
A6	Tools named (for example GitLab CI, SonarQube, Terraform, Prometheus, Grafana) are examples only	Any equivalent tools can be used; the practices stay the same
A7	The timeline of 18 months is an estimate	Real durations depend on size, skills and legacy

5.3 Constraints

- **Regulation and audit:** every production change must be traceable, approved and evidenced.
- **Segregation of duties:** the person who writes a change should not be the only one to approve it into production.
- **Availability:** customers expect banking services around the clock; deployments must cause little or no downtime.
- **Security and data protection:** customer data must stay protected throughout the pipeline and in test environments.
- **Legacy technology:** older systems limit how fast some parts can be automated.
- **People:** limited DevOps skills and the risk of change fatigue.
- **Budget and time:** tooling, training and effort are limited.
- **Scope:** this is a design and planning exercise; no real systems are changed.

6. Proposed Solution: DevOps Transformation Roadmap

The solution has eight parts. The assignment's required topics (collaboration, automation, CI/CD, quality gates, monitoring, feedback loops and culture) are Sections 6.2 to 6.8.

6.1 Vision and Guiding Principles

Target state: small changes flow from a developer's computer to production through an automated, secure and monitored pipeline, owned by one cross-functional team.

The transformation follows the **CALMS** framework, a simple checklist for DevOps:

Letter	Principle	Simple meaning	Bank example
C	Culture	Shared ownership and trust; blame the process, not people	Dev and Ops share one on-call rota
A	Automation	Let machines do repetitive work	Automated tests and deployments
L	Lean	Small batches, less waiting, less waste	Release one feature at a time, not 200
M	Measurement	Decide using data	Track release failures and recovery time
S	Sharing	Share knowledge and tools openly	Shared dashboards and a team wiki

Order of importance: culture first, then practices, then tools. Tools alone do not create DevOps.

6.2 Collaboration Practices

Practice	What it means	Problem it fixes
Cross-functional teams	Developers, testers, operations and security work in one team around a product	Hand-offs and delays (C4)
Shared ownership	The team that builds a service also helps run it ("you build it, you run it"), introduced gradually	Nobody owning the whole outcome
Shared goals and backlog	One list of work and common targets for the team	Teams pulling in different directions
Shift left	Involve testing, security and operations from the design stage, not the end	Late discovery of defects and vulnerabilities (C2, C8)
Daily stand-ups and joint planning	Short daily check-ins; planning together	Surprises and poor communication
Shared tools and chat	One place for code, tickets, pipeline results and alerts	Information hidden in silos
Blameless postmortems	After an incident, examine what in the system allowed it	Fear and hiding problems
Definition of Done	"Done" includes tests, monitoring and documentation	Features that cannot be supported

6.3 Automation Opportunities

Rule of thumb: automate first what is **painful and frequent**. Quick wins build trust for the harder work.

Manual activity today	Automate with	Benefit	Priority
Building the application	Automated build on every change	Consistent, fast, repeatable	High
Unit and regression testing	Automated test suites in the pipeline	Bugs found in minutes, not months	High
Deploying to environments	Scripted, automated deployment	No human error; repeatable	High
Security and quality checks	Code scans, dependency scans, secret detection	Problems caught early	High
Setting up servers and environments	Infrastructure as Code	Identical environments on demand	Medium
Database changes	Versioned, automated migrations	Safer data changes	Medium
Collecting audit evidence	Pipeline records who, what, when and test results	Compliance without paperwork	Medium
Rollback	One-click or automatic rollback	Fast recovery	Medium

Manual activity today	Automate with	Benefit	Priority
Monitoring and alerts	Automated dashboards and alerting	Problems seen before customers report them	Medium
Release notes and change tickets	Generated from the pipeline and tickets	Saves time; fewer mistakes	Low

Not automated at first: exploratory testing (human curiosity finds unexpected problems) and risk judgement for high-impact changes. Automation supports people; it does not replace thinking.

6.4 CI/CD Concepts and Pipeline

Term	Meaning	Fit for this bank
Continuous Integration (CI)	Developers merge small changes often; each merge is built and tested automatically	Adopt for all teams
Continuous Delivery	Code is always ready to release; a person approves the final step to production	**Recommended** for the bank, because approvals and audit are required
Continuous Deployment	Every change that passes all checks goes to production automatically	Consider later, only for low-risk applications

Key principle: build once, deploy many. The same tested package is promoted through each environment (Test, Staging, Production), so what was tested is exactly what is released.

Pipeline stages

Stage	What happens	Example tool
1. Commit	Developer pushes a small change to version control	GitLab
2. Build	Code is compiled and packaged into one artifact	GitLab CI
3. Test	Unit tests and static code analysis run	Test frameworks, SonarQube
4. Security scan	Dependency, code and secret scans	GitLab security scanners
5. Deploy to Test	Artifact deployed automatically; integration and API tests run	Deployment scripts, Terraform
6. Deploy to Staging	Production-like environment; performance and acceptance tests	Same scripts
7. Approval	Risk check and sign-off; evidence attached automatically	GitLab approvals, change tool
8. Deploy to Production	Gradual release with health checks	Deployment scripts
9. Monitor	Dashboards and alerts watch the release	Prometheus, Grafana

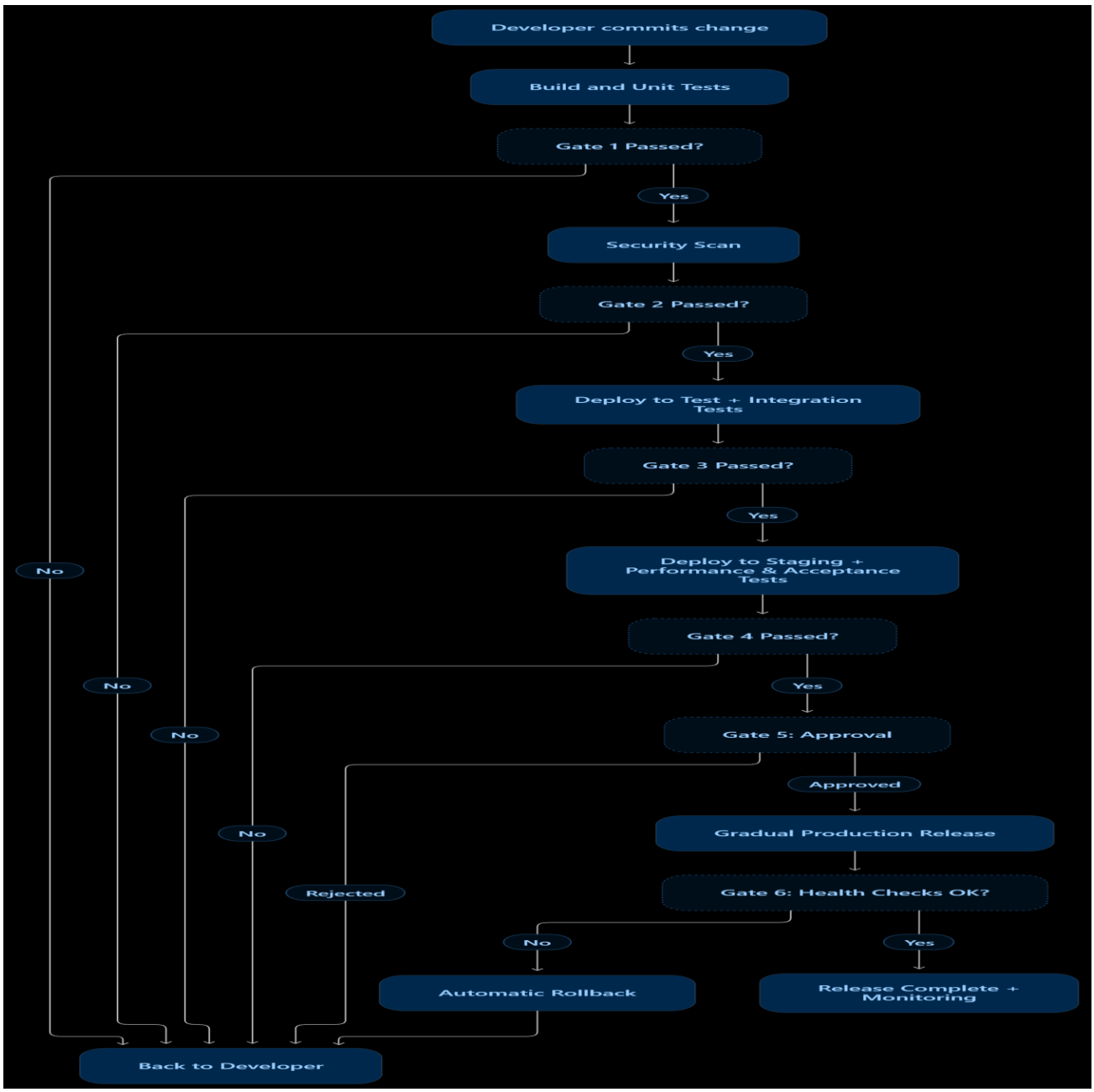


DIAGRAM 2: CI/CD PIPELINE WITH QUALITY GATES

Illustrative pipeline definition

```
# .gitlab-ci.yml (simplified example)
stages: [build, test, security, deploy_test, deploy_staging, deploy_prod]

build:
  stage: build
  script: ./build.sh

unit_tests:
  stage: test
  script: ./run_unit_tests.sh

security_scan:
  stage: security
  script: ./run_security_scan.sh

deploy_staging:
  stage: deploy_staging
  script: ./deploy.sh staging
  environment: staging
```

```
deploy_prod:
  stage: deploy_prod
  script: ./deploy.sh production
  environment: production
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual            # human approval gate
```

Safer ways to release

Strategy	How it works	Benefit
Blue-green deployment	Two identical environments; switch customers from old to new	Near-zero downtime; instant rollback
Canary release	Release to a small share of users first, then widen	Problems affect few customers
Feature flags	Deploy code switched off, turn it on later	Separates deployment from release
Automated rollback	Pipeline reverts when health checks fail	Fast recovery

6.5 Quality Gates

Simple explanation: a quality gate is an automatic checkpoint. If the change does not meet the rule, the pipeline stops and the change goes no further. This replaces end-of-project inspection with checks at every step.

Gate	Where	Example criteria (illustrative)	Automated?
G1 Build and unit	After commit	Code builds; unit tests pass; test coverage at least 80%; no critical code issues	Yes
G2 Security	After build	No critical or high vulnerabilities; no secrets in code; licences acceptable	Yes
G3 Test environment	After deploy to Test	Integration and API tests pass; database migration succeeds	Yes
G4 Staging	After deploy to Staging	Regression passes; performance within limits; user acceptance sign-off	Mostly
G5 Release approval	Before production	Change record created automatically; risk reviewed; approver signs off	Partly (human sign-off)
G6 Post-deployment	After production release	Health checks pass; error rate and speed normal; automatic rollback if not	Yes

Rule: stop the line. A failed gate stops everything until it is fixed, and fixing it is the team's top priority.

6.6 Monitoring Strategy

Simple explanation: monitoring means watching the system continuously so the team knows about a problem before customers do. More frequent releases are only safe if problems are seen and fixed quickly.

Layer	What to watch	Examples
Infrastructure	Health of servers and networks	CPU, memory, disk, network
Application	How the software behaves (the "four golden signals")	Error rate, response time, traffic, saturation
Business	Whether customers can complete banking tasks	Payment success rate, login failures, failed transfers
Security	Signs of attack or misuse	Repeated failed logins, unusual access patterns

- **Three data types:** metrics (numbers over time), logs (event records) and traces (a request's journey through services).
- **Service level objectives (SLOs):** agreed targets such as 99.9% availability for internet banking; used to judge whether releases are healthy.
- **Alerting:** alert on customer-visible symptoms, set severity levels, and route to an on-call person. Avoid alert overload.
- **Incident response:** detect, respond, recover, then run a blameless postmortem; findings feed back into the backlog.
- **Dashboards:** shared by Dev, Ops and business so everyone sees the same facts.
- **Data protection:** logs must not contain customer personal data or secrets.

6.7 Feedback Loops

Simple explanation: a feedback loop is how information about results returns to the people who can act on it. The shorter the loop, the cheaper the fix.

Loop	Speed	Feedback from to	Example
Automated tests	Minutes	Pipeline to developer	A failed unit test after a commit
Code review	Hours	Reviewer to developer	Merge request comments
Staging and acceptance	Hours to days	Testers and users to team	Acceptance test finds a missing step
Production monitoring	Real time	Operations data to team	Alert after a canary release
Customer feedback	Days to weeks	Customers to product owner	Survey, app reviews, complaints
Retrospectives and postmortems	Weekly to monthly	Team to itself and leadership	Improvement actions after an incident
Business metrics	Monthly	Results to leadership	Release metrics reviewed against targets

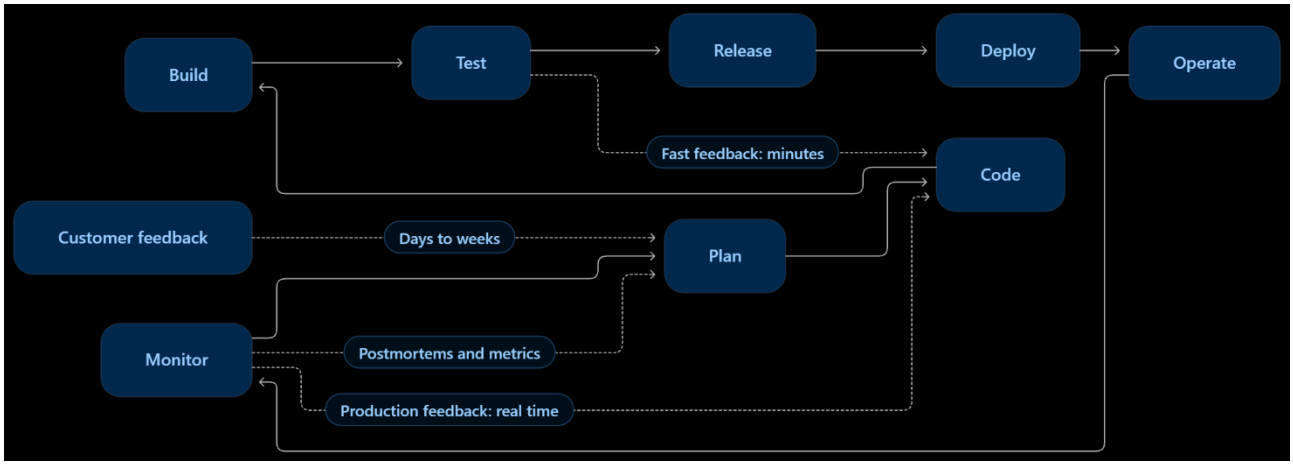


DIAGRAM 4: FEEDBACK LOOPS

6.8 Cultural Changes

Culture is the hardest and most important part. These shifts are needed:

From	To
Blame the person who made the mistake	Blame the process; learn through blameless postmortems
Separate teams passing work along	One team with shared ownership

From	To
Release is a risky, rare event	Release is a routine, boring, frequent event
Quality checked at the end by QA	Quality built in by everyone, at every step
Control through approvals and paperwork	Control through automated checks and evidence
Fear of failure; avoid change	Small experiments; learn quickly
Knowledge held by a few experts	Knowledge shared in writing and in the open

How to make culture change happen

- **Visible leadership support:** leaders explain why and model the behaviour (for example, no blame after incidents).
- **Training and coaching:** short courses, pairing, and DevOps champions inside each team.
- **Pilot and share success:** a real win in the pilot is the best argument.
- **Align incentives:** measure teams on shared outcomes (stability and delivery), not on separate goals.
- **Time to learn:** protect time for training and improvement.

7. Implementation Roadmap

The roadmap runs over about **18 months** (an assumption). It starts small, proves value, then scales.

Why a pilot instead of a big-bang change? A bank cannot afford to risk everything at once. A pilot on a lower-risk application (for example a notification service or customer portal, not core banking) lets the bank learn, fix mistakes cheaply, and gather proof to persuade doubters.

Phase	Months	Activities	Output
1. Foundation	1 to 3	Map the current release process; measure baselines; choose the pilot application; form a cross-functional pilot team; agree Git workflow and standards; leadership alignment; initial training	Baseline metrics; pilot team; agreed standards
2. Automate	4 to 6	Build the CI pipeline; automate builds and unit tests; add code and security scans; create test environments as code; start basic monitoring	Working CI for the pilot; first automated tests
3. Deliver	7 to 12	Add automated deployment to Test and Staging; automated regression and security tests; quality gates G1 to G6; approval step with automatic evidence; canary releases and rollback; dashboards and alerts; onboard 2 to 3 more applications	Pilot releasing monthly then fortnightly; several apps on the pipeline
4. Scale and optimise	13 to 18	Extend to remaining suitable applications; define SLOs; run incident practice drills; hold communities of practice; review metrics and improve	Most digital applications on the pipeline; continuous improvement habit

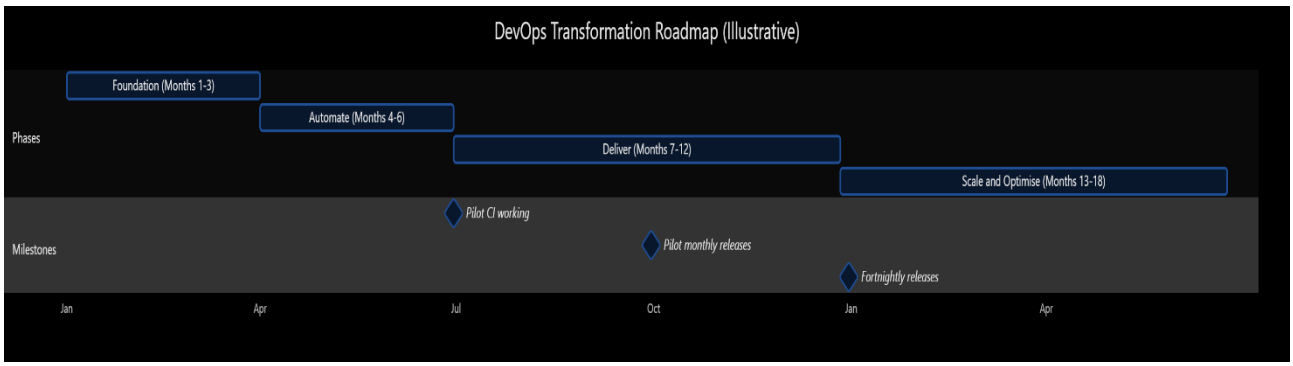


DIAGRAM 4: TRANSFORMATION ROADMAP

7.1 Release Cadence Milestones (illustrative)

Milestone	Target state
Start	Every six months, manual testing and deployment
Month 6	Pilot application builds and tests automatically on every change
Month 9	Pilot application released monthly through the pipeline
Month 12	Pilot released every two weeks; 2 to 3 more applications monthly
Month 18	Most digital channel applications released every two weeks or better

8. Measurable Success Criteria

The first four measures are the widely used **DORA metrics**, which describe software delivery performance. Only the six-month release cycle comes from the assignment. Other baselines are assumed (Assumption A5), and targets are illustrative.

#	Metric	Baseline	Target	How measured
S1	Deployment frequency	Every 6 months (given)	Monthly by month 12; fortnightly by month 18 (digital channels)	Pipeline records
S2	Lead time for changes (commit to production)	Months (assumed)	Under 1 week for standard changes	Pipeline and ticket timestamps
S3	Change failure rate	High (assumed 25 to 30%)	Under 15%	Incident and rollback records
S4	Time to restore service	Days (assumed)	Under 4 hours	Incident log
S5	Regression tests automated	Low (assumed under 20%)	At least 70%	Test management tool
S6	Manual steps in deployment	Many (assumed)	None, except the approval	Deployment runbook review
S7	Production changes with full audit trail	Partial (assumed)	100%	Audit sample checks
S8	Commit-stage feedback time	Not available	Under 15 minutes	Pipeline metrics
S9	Release-related incidents	Baseline measured in Phase 1	50% fewer within 12 months	Incident log
S10	Teams reporting better collaboration	Survey baseline in Phase 1	Clear improvement in repeat survey	Staff survey

9. Recommendations and Lessons Learned

9.1 Recommendations

1. Start with culture and a pilot team on a low-risk application, not with tool purchases.
2. Automate testing first. A bank cannot release safely faster than it can verify the release.
3. Build compliance and security into the pipeline so evidence is produced automatically.
4. Invest in monitoring and rollback before increasing release frequency.
5. Make changes small. Smaller releases are easier to test, understand and undo.
6. Measure the DORA metrics from day one and share results openly.
7. Treat legacy and core banking systems as a separate, slower track.

9.2 Lessons Learned

These come from designing this plan, not from a live transformation.

- **Culture comes before tools.** Tools without shared ownership only automate the old problems.
- **Smaller is safer.** Releasing more often sounds riskier, but small changes are easier to control.
- **Automating a bad process makes it fail faster.** Simplify the process first, then automate it.
- **Speed and compliance are not opposites.** Automatic evidence can make audits easier, not harder.
- **Prove it small first.** A pilot with real results is more persuasive than a presentation.
- **What gets measured improves.** Baselines turn opinions into evidence.

9.3 Limitations of This Plan

- Baselines, timeline and team sizes are assumptions; a real bank would measure them first.
- Costs and staffing levels are not estimated.
- Core banking and mainframe modernisation is only outlined, not designed in detail.
- Tool names are examples; a real selection would compare options against the bank's needs.
- Cultural change is hard to schedule; the 18-month timeline could easily slip.